

Методические указания 16

Spring AOP

[Аспектно-ориентированное программирование \(AOP\)](#)

[Основные понятия](#)

[Spring AOP](#)

[Пример](#)

[Практическое задание](#)

[Дополнительные материалы](#)

[Используемая литература](#)

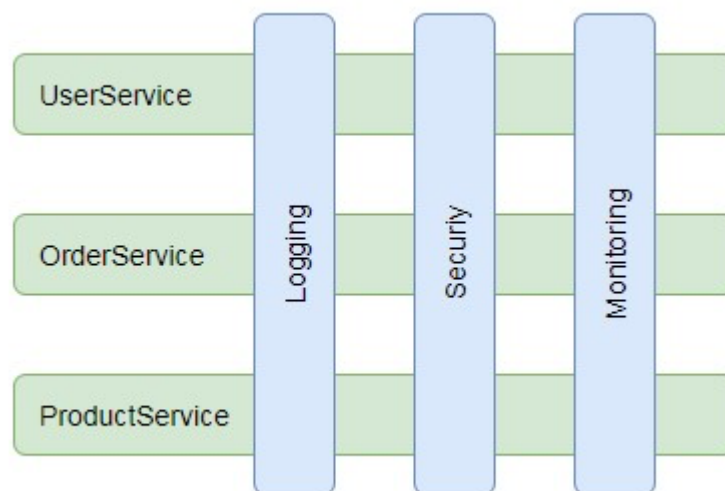
Аспектно-ориентированное программирование (AOP)

Аспектно-ориентированное программирование (АОП) **дополняет** объектно-ориентированное (ООП), предоставляя еще один способ мышления о структуре программы. Ключевым элементом модульности в ООП является класс, тогда как в АОП это аспект. АОП **не является** частью ООП, а предлагает новые возможности и позволяет элегантно решать проблемы, возникающие в узких местах классического ООП.

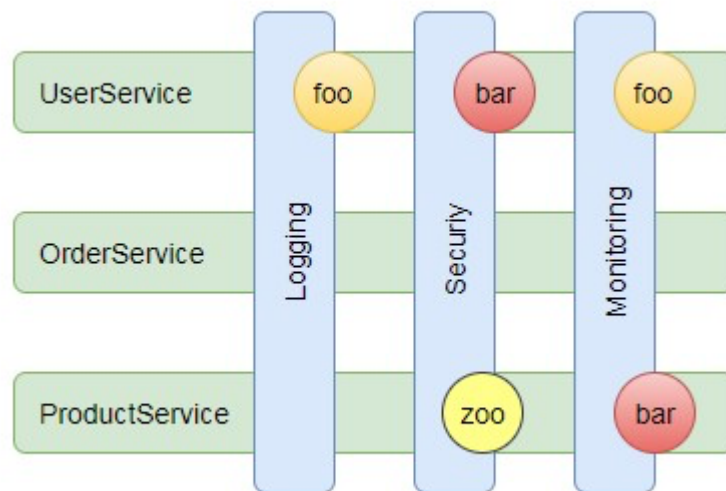
Рассмотрим пример: есть набор не связанных сервисов (например, **UserService** , **OrderService** , **ProductService**).



Надо реализовать функциональность, не имеющую ничего общего с этим набором сервисов, но оказывающую воздействие на них одинаково или, возможно, по-разному — в зависимости от конкретного сервиса. Например, логирование, управление доступом или мониторинг.



В общем виде это действие является суперпозицией сервисов и добавляемой функциональности. Причем информацией о том, на каком пересечении применить то или иное действие или не применять его вовсе, не обладают ни сервисы, ни добавляемая функциональность.



Таким образом в системе появляется третий компонент, который:

- «знает», где и какое именно действие применить;
- «знает» эти действия, то есть содержит так или иначе их код;
- элементы, над которыми он стоит, могут и не догадываться о его существовании.

Этот компонент и является аспектом в АОП.

Основные понятия

Аспект (Aspect) — модуль или класс, реализующий сквозную функциональность. Аспект изменяет поведение остального кода, применяя «совет» в «точках соединения», определенных некоторым «срезом».

Срез (Pointcut) — набор «точек соединения». Срез определяет, подходит ли данная точка соединения к данному «совету».

Точка соединения (Join-Point) — точка в выполняемой программе, где следует применить «совет».

Совет (Advice) — средство оформления кода, которое должно быть вызвано из точки соединения. Совет может быть выполнен до, после или вместо точки соединения.

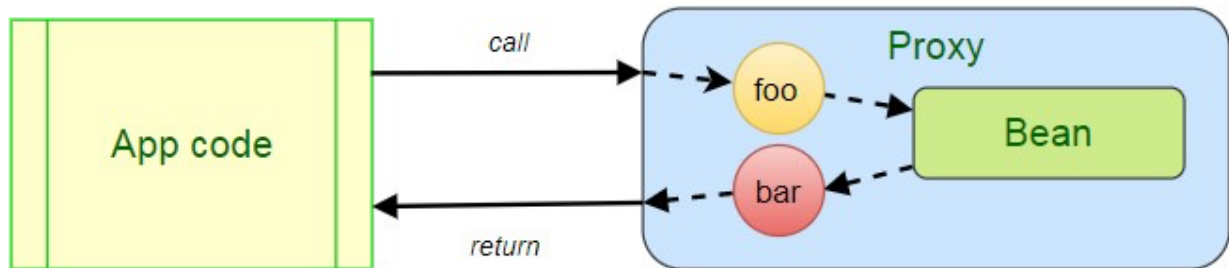
Внедрение (introduction) — изменение структуры класса и/или изменение иерархии наследования для добавления функциональности аспекта в инородный код.

Таким образом, **Аспект**, обладая информацией (получаемой от **советов**), разрывает выполнение программного кода аспектируемых классов в **точке соединения** и **внедряет** туда известную ему функциональность (программный код).

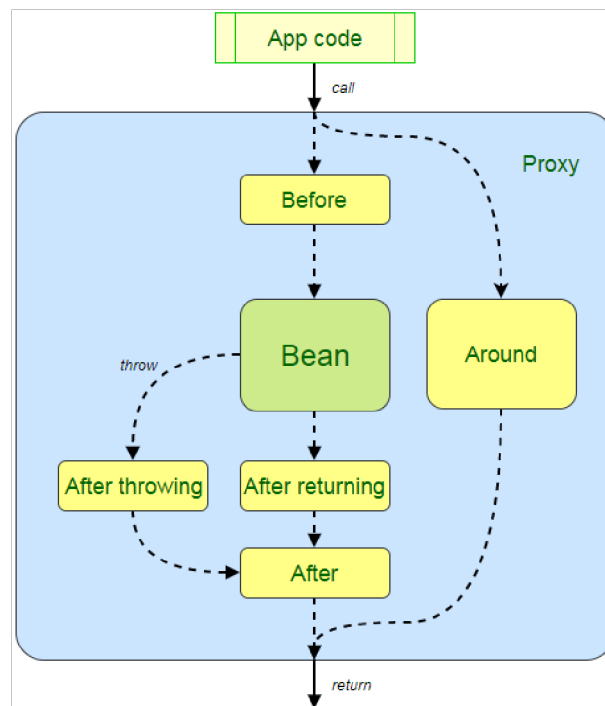
Spring AOP

«Из коробки» Java не поддерживает АОП. Для этого существуют сторонние библиотеки, например AspectJ. Spring применяет собственную реализацию АОП — Spring AOP, которая использует стиль описания AspectJ, но не реализует все ее возможности.

Для внедрения кода Spring AOP применяет проксирование бинов, находящихся в IoC-контейнере. Это избавляет как от использования специальных загрузчиков классов (**classloaders**), так и от посткомпиляции или работы со специальным компилятором, что в целом соответствует «легковесной» направленности Spring.



Фреймворк создает прокси-классы для тех бинов, на которые ссылаются определенные конфигурацией Pointcut'ы. Ограничение такого подхода — невозможность создать аспект для другого аспекта.



Spring AOP поддерживает советы (advice) по выполнению кода до метода бина, после возврата из него, после выброса исключения, после последних двух и вместо выполнения метода бина.

Пример

Рассмотрим тестовый пример. Создадим приложение, которое логирует обращения ко всем собственным сервисам.

Spring AOP подключается добавлением зависимости:

```
<dependency>
  <groupId> org.aspectj </groupId>
  <artifactId> aspectjweaver </artifactId>
  <version> LATEST </version>
</dependency>
```

Для **Spring Boot** добавляется стартером:

```
<dependency>
  <groupId> org.springframework.boot </groupId>
  <artifactId> spring-boot-starter-aop </artifactId>
</dependency>
```

Spring AOP поддерживает как конфигурирование, основанное на аннотациях, так и XML-конфигурирование.

Включение автопроксирования бинов:

```
@Configuration
@EnableAspectJAutoProxy
```

Или аналогичный способ:

```
<aop:aspectj-autoproxy/>
```

Сервис:

```
package com . example . aop .
service;
@Service
public class SomeService {
    private String name = SomeService . class . getCanonicalName ();
    // что-то полезное
    public String getName () {
return name;
    }
}
```

Обратите внимание на пакет **com.example.aop.services** . В дальнейшем мы будем ссылаться на него в определении среза (pointcut) аспекта.

Аспект:

```
@Aspect
```

```

@Component
public class LogAspect {
    // определяем срез по всем методам бинов из пакета com.example.aop.service
    @Pointcut ("execution(* com.example.aop.service..*.*(..))")
    private void getName () {
    }
    // определяем совет (Advice) "ПЕРЕД" выполнением кода бина (класса)
    @Before ("getName()")
    public void logBefore (JoinPoint joinPoint ) {
        // выводим в консоль информацию о текущей точке соединения
        System . out . println (joinPoint      );
    }
}

```

При обращении к методам сервисов, например, в контроллере:

```

@RestController
public class SomeController {
    @RequestMapping ("/")      public
    String getGreeting () {
        return someService . getName ();
    }
    // ...
}

```

На консоль будут выдаваться сообщения, определенные в методе аспекта **logBefore(JoinPoint joinPoint)** :

```

execution (String com . example . aop . service . SomeService . getName ())

```

Обобщим материал.

1. Есть сервисы, которые не знают о существовании сторонней функциональности (в данном случае — логирования), их код не подвергался дополнительным модификациям (относительно типичного сервиса).
2. Имеется сторонняя функциональность (логирование), которая готова работать с любым объектом.
3. Определенный нами аспект, используя возможности библиотеки Spring AOP, меняет поведение исходных объектов (сервисов), заставляя выполняться при определенных нами условиях отдельный код, не пересекающийся с кодом исходных объектов.

Следует отметить, что функциональность Spring Security основана на применении АОП.

Например, аннотация метода **@PreAuthorize(...)** гарантирует, что только пользователь, обладающий определенными правами, имеет доступ к аннотируемому методу. Spring Security будет использовать во время запуска (AOP) pointcut для запуска совета (**advice**) **Before** на методах, помеченных данной аннотацией. И будет выбрасывать **AccessDeniedException** , если ограничения безопасности, указанные в параметрах аннотации, не будут выполнены.

Практическое задание

1. Только пользователи с ролью ADMIN должны иметь возможность добавлять/редактировать товары;
2. Подсчитать какие три товара наиболее часто просматривают пользователи;
3. Сделать так, чтобы логин/пароль можно было “вбивать” прямо на странице с товарами;
4. После аутентификации, где-нибудь на странице должно быть вывешено имя пользователя;

Дополнительные материалы

1. Кей С. Хорстманн, Гари Корнелл Java. Библиотека профессионала. Том 1. Основы // Пер. с англ. — М.: Вильямс, 2014. — 864 с.
2. Брюс Эккель. Философия Java // 4-е изд.: Пер. с англ. — СПб.: Питер, 2016. — 1 168 с.
3. Г. Шилдт. Java 8. Полное руководство // 9-е изд.: Пер. с англ. — М.: Вильямс, 2015. — 1 376 с.
4. Г. Шилдт. Java 8: Руководство для начинающих. // 6-е изд.: Пер. с англ. — М.: Вильямс, 2015. — 720 с.

Используемая литература

Для подготовки данного методического пособия были использованы следующие ресурсы:

1. <https://docs.spring.io/spring/docs/2.5.x/reference/aop.html>